UNIVERSIDAD NACIONAL DEL ALTIPLANO PUNO

FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

ESCUELA PROFESIONAL INGENIERÍA DE SISTEMAS



CURSO:

Programación Orientada a Objetos-II

TRABAJO:

PRESENTADO POR:

Herencia y Polimorfismo — Jerarquías, Sobreescritura y Principio de Sustitución

Melvin Ronald Choque Chaiña

DOCENTE:

Mg. Aldo Hernan Zanabria Galvez

FECHA:

13/09/2026

PUNO-PERÚ

Contenido

1. Objetivo General 3

1,1, Objetivos Específicos 3

2. Marco Teórico 3

2.1. Herencia — IS-A vs HAS-A 3

2.2. Polimorfismo Dinámico y Sobreescritura 4

Python 3.12 4

C++17 4

2.3. Principio de Sustitución de Liskov (LSP) 5

3. Materiales 5

4. Instrucciones –PC1 6

5. Desarrollo de actividades 6

5.1. Actividad 1 — Análisis — Identificar IS-A vs HAS-A 6

5.2. Actividad 2 — Jerarquía de Personal — Python con Polimorfismo 8

5.3. Actividad 3 — Verificación del LSP — Test de sustitución 10

5.4. Actividad 4 — Jerarquía en C++17 — virtual, override y destructor 12

5.5. Actividad 5 — Proyecto Integrador — Jerarquía de herencia del sistema 14

6. Diagrama UML 15

7. Preguntas de Reflexión 15

1. Objetivo General

Diseñar e implementar jerarquías de herencia correctamente justificadas en Python y C++17, aplicando sobreescritura de métodos, polimorfismo dinámico y el principio de sustitución de Liskov (LSP), avanzando en el Proyecto Integrador del semestre.

1,1, Objetivos Específicos

- Identificar cuando la herencia está justificada (relación IS-A) frente a cuando debe preferirse la composición (HAS-A).
- Implementar jerarquías de herencia Python usando super() y en C++ con herencia pública y virtual.
- Sobrescribir métodos @overrride (Python 3.12+) y override (C++17), comprendiendo el mecanismo de vtable
- Verificar el Principio de Sustitución de Liskov: todo objeto de una subclase debe poder sustituir a su clase base sin romper el comportamiento esperado.
- Extender el Proyecto Integrador con una jerarquía de herencia bien justificada

2. Marco Teórico

2.1. Herencia — IS-A vs HAS-A

La herencia modela la relación “ES UN” (IS-A): un Docenre ES UN PersonalUniversitario. La composición modela “TIENE UN” (HAS-A): un Docente TIENE UN Departamento. El error más frecuente es usar herencia cuando corresponde composición, creando jerarquías frágiles y difíciles de mantener (Larman, 2004):

```python
# ☑ Herencia justificada — IS-A
class PersonalUniversitario:
 def __init__(self, dni: str, nombres: str):
 self._dni = dni
 self._nombres = nombres
 def identificarse(self) -> str:
 return f"{self._nombres} (DNI: {self._dni})"
class Docente(PersonalUniversitario): # Docente ES
PersonalUniversitario
 def __init__(self, dni, nombres, categoria: str):
 super().__init__(dni, nombres)
 self._categoria = categoria
 def calcular_bonificacion(self) -> float: ...
```

```
# ❌ Herencia NO justificada — misma interface, distintos dominios
class Automovil(Motor): # Automovil NO es un Motor
    pass # debería ser composicion: self._motor = Motor()
```

2.2. Polimorfismo Dinámico y Sobreescritura

El polimorfismo dinámico permite que una referencia de tipo base invoque el método de la subclase real en tiempo de ejecución. En Python esto es automático; en C++ requiere declarar el método como virtual en la base y override en las subclases:

Python 3.12

```python
# Python — polimorfismo dinámico automático
class PersonalUniversitario:
 def calcular_remuneracion(self) -> float:
 raise NotImplementedError("Subclase debe implementar este método")
class Docente(PersonalUniversitario):
 def calcular_remuneracion(self) -> float:
 return self._horas * self._tarifa * self._factor_categoria()
class Administrativo(PersonalUniversitario):
 def calcular_remuneracion(self) -> float:
 return self._salario_base + self._bono_asistencia
# Lista heterogénea — polimorfismo en acción
personal: list[PersonalUniversitario] = [
 Docente("12345678", "Mamani", 16, "Asociado"),
 Administrativo("87654321", "Quispe", 2800.0),
]
for p in personal:
 print(f"{p.identificarse()}: S/ {p.calcular_remuneracion():.2f}")
```

C++17

```cpp
// C++17 — virtual + override
class PersonalUniversitario {
protected:
 std::string dni_;
 std::string nombres_;
public:
 PersonalUniversitario(std::string dni, std::string nombres)
 : dni_(std::move(dni)), nombres_(std::move(nombres)) {}
 virtual ~PersonalUniversitario() = default; // OBLIGATORIO si hay virtual
 virtual float calcularRemuneracion() const = 0; // método virtual puro
 virtual std::string identificarse() const {
 return nombres_ + " (DNI: " + dni_ + ")";
 }
};
class Docente : public PersonalUniversitario {
 int horas_;
 float tarifa_;
public:
 Docente(std::string dni, std::string nombres, int horas, float tarifa)
 : PersonalUniversitario(std::move(dni), std::move(nombres)),
 horas_(horas), tarifa_(tarifa) {}
```

```
22  float calcularRemuneracion() const override {
23  return horas_ * tarifa_;
24  }
25 };
```

2.3. Principio de Sustitución de Liskov (LSP)

El Principio de Sustitución de Liskov establece que, si S es subtipo de T, entonces los objetos de tipo T en un programa pueden ser reemplazados por objetos de tipo S sin alterar ninguna propiedad deseable del programa (Liskov, 1987). En términos prácticos: una subclase no debe debilitar las precondiciones ni fortalecer las postcondiciones de la clase base:

```
 1 # VIOLACIÓN del LSP — la subclase cambia el contrato
 2 class Rectangulo:
 3  def set_ancho(self, v): self._ancho = v
 4  def set_alto(self, v): self._alto = v
 5  def area(self): return self._ancho * self._alto
 6 class Cuadrado(Rectangulo): # Cuadrado NO es Liskov-sustituto de Rectangulo
 7  def set_ancho(self, v):
 8  self._ancho = v
 9  self._alto = v # efecto secundario inesperado
10 # Código que funciona con Rectangulo pero FALLA con Cuadrado
11 def verificar_area(r: Rectangulo):
12  r.set_ancho(4)
13  r.set_alto(5)
14  assert r.area() == 20 # falla si r es Cuadrado (area = 25)
```

3. Materiales

Herramientas	Uso
Python 3.12+/Colab	Actividades 1-3; verificación de LSP con assert
g++ -std=c++17 -Wall	Actividades 3-4; jerarquía con virtual y override
Draw.io / StarUML	Diagrama de herencia del Proyecto integrador (Act 5)
GitHub	Commit:”feat(semana3): jerarquía de herencia”

4. Instrucciones –PC1

⚠	**Esta es la Práctica Calificada 1 (PC1)** La nota de esta sesión corresponde a la Práctica Calificada 1 del semestre (peso: 30% dentro de la Unidad I). Trabaje de forma individual. Al finalizar, suba su notebook a Google Classroom y haga commit al repositorio GitHub. Tiempo: 2 horas.

1. Abrir el notebook de Colab de las semanas anteriores y agregar una sección "Semana 3".
2. Completar las actividades en orden; cada actividad tiene sus criterios de calificación explícitos.
3. El código debe ejecutarse sin errores. Los bloques de TODO deben estar completamente implementados.
4. La actividad 5 se continua fuera de clase: commit al repositorio antes del siguiente lunes.

5. Desarrollo de actividades

5.1. Actividad 1 — Análisis — Identificar IS-A vs HAS-A

Para cada relación propuesta, indique si debe implementarse con HERENCIA (IS-A) o con COMPOSICIÓN (HAS-A), justificando brevemente. Si sugiere herencia, especifique la clase base y la subclase.

Relación propuesta	IS-A / HAS-A	Justificación
Un estudiante de Posgrado en relación a un Estudiante	IS-A	Un Estudiante de Posgrado ES un Estudiante, porque comparte los mismos atributos y comportamientos básicos (nombre, matrícula, cursos inscritos, etc), pero añade características específicas (ej. Tesis, nivel académico, requisitos adicionales).

		Por eso la subclase EstudiantePosgrado puede heredar de la clase Estudiante y ser usada en cualquier contexto donde se espera un Estudiante.
Un Sensor loT en relación a un Dispositivo	IS-A	Un Sensor loT ES un Dispositivo, porque comparte características básicas que cualquier dispositivo (identificador, conexión, energía, etc) y además añade funcionalidades específicas como capturar datos ambientales y transmitirlos por red. Por lo tanto, la herencia está justificada: SonsorLoT puede ser una subclase de Dispositivo y usarse en cualquier contexto donde se espera un Dispositivo.
Un Informe en relación a una Impresora	HAS-A	Un Informe NO es una impresora; son entidades de dominios distintos. El informe es un documento, mientras que la impresora es un dispositivo. Los que ocurre es que un Informe PUEDE SER IMPRESO usando una Impresora, es decir, el Informe tiene una relación de uso o dependencia con la impresora. Por eso se modela como composición: un objeto Informe puede tener una referencia a una Impresora para ejecutar la acción de imprimir, pero no hereda de ella.
Un Libro Digital en relación a un Libro	IS-A	Un Libro Digital ES un Libro, porque comparte las mismas características de un libro (título, autor, contenido, capítulos) y simplemente cambia de formato en que se presenta. Por lo tanto, la herencia está justificada: LibroDigital puede ser una subclase de Libro y usarse en cualquier contexto donde se espera un Libro.

Una Motocicleta en relación a un Vehículo	IS-A	Una Motocicleta ES un Vehiculo, porque comparte las características esenciales de cualquier vehículo (medio de transporte, velocidad, capacidad de desplazamiento, matricula, etc), y simplemente es una especialización con atributos propios (motor de dos ruedas, casco obligatorio, etc.). Por lo tanto, la herencia está justificada: Motocicleta puede ser una subclase de Vehículo y usarse en cualquier contexto donde se espera un Vehículo.
Un Departamento en relación a un Empleado	HAS-A	Un Empleado NO es un Departamento, son entidades distintas. Lo que ocurre es que un empleado TIENE un Departamento asociado, es decir, pertenece o está adscrito a uno. Por eso la relación se modela como Composición: la clase Empleado puede tener un atributo departamento, pero no heredar de Departamento.

5.2. Actividad 2 — Jerarquía de Personal — Python con Polimorfismo

Complete la implementación de la jerarquía de personal universitario. La clase base PersonalUniversitario ya está implantada; usted debe completar las subclases Docente y Administrativo. Luego implemente la función planilla() que demuestra el polimorfismo.

```
1 # ── Práctica S3 — Actividad 2: Jerarquía PersonalUniversitario ──────────
2 from abc import ABC, abstractmethod
3 from dataclasses import dataclass
4 class PersonalUniversitario(ABC):
5  """Clase base abstracta para todo el personal de la UNAP."""
6  TARIFA_HORA_BASE = 75.0 # S/ por hora
7  def __init__(self, dni: str, nombres: str, apellidos: str):
8  self.__dni = self._validar_dni(dni)
```

```
 9  self._nombres = nombres
10  self._apellidos = apellidos
11  @staticmethod
12  def _validar_dni(dni: str) -> str:
13  if not (dni.isdigit() and len(dni) == 8):
14  raise ValueError(f"DNI inválido: {dni!r} (debe tener 8 dígitos)")
15  return dni
16  @property
17  def dni(self) -> str: return self.__dni
18  @property
19  def nombre_completo(self) -> str:
20  return f"{self._apellidos}, {self._nombres}"
21  @abstractmethod
22  def calcular_remuneracion(self) -> float:
23  """Cada subclase define su propio cálculo."""
24  ...
25  def __str__(self) -> str:
26  return f"{self.nombre_completo} → S/ {self.calcular_remuneracion():.2f}"
27 class Docente(PersonalUniversitario):
28  FACTORES = {"Auxiliar": 1.0, "Asociado": 1.4, "Principal": 1.8}
29  def __init__(self, dni: str, nombres: str, apellidos: str,
30  horas: int, categoria: str):
31  super().__init__(dni, nombres, apellidos)
32  # TODO: validar horas > 0 y <= 40
33  # TODO: validar categoria en FACTORES
34  # TODO: asignar self._horas y self._categoria
35  ...
36  def calcular_remuneracion(self) -> float:
37  # TODO: horas * TARIFA_HORA_BASE * factor de categoria
38  ...
39 class Administrativo(PersonalUniversitario):
40  def __init__(self, dni: str, nombres: str, apellidos: str,
41  salario_base: float, bono: float = 0.0):
42  super().__init__(dni, nombres, apellidos)
43  # TODO: validar salario_base > 0
44 # TODO: asignar self._salario_base y self._bono
45  ...
46  def calcular_remuneracion(self) -> float:
47  # TODO: salario_base + bono
48  ...
49 class Investigador(PersonalUniversitario):
50  """Investigador a tiempo completo — remunerado por proyectos activos."""
51  REMUNERACION_POR_PROYECTO = 1500.0
52  def __init__(self, dni: str, nombres: str, apellidos: str,
53  proyectos: int):
54  super().__init__(dni, nombres, apellidos)
55  if proyectos < 1:
56  raise ValueError("Debe tener al menos 1 proyecto activo")
57  self._proyectos = proyectos
58  def calcular_remuneracion(self) -> float:
59  return self._proyectos * self.REMUNERACION_POR_PROYECTO
60 def planilla(personal: list[PersonalUniversitario]) -> None:
61  """Imprime la planilla usando polimorfismo — sin isinstance()."""
62  total = 0.0
63  print("=" * 60)
64  print(f"PLANILLA PERSONAL UNIVERSITARIO — UNAP PUNO")
65  print("=" * 60)
```

```
66  for p in personal:
67  print(p) # llama __str__ → calcular_remuneracion polimórfico
68  total += p.calcular_remuneracion()
69  print("-" * 60)
70  print(f"TOTAL PLANILLA: S/ {total:.2f}")
71 # ── Pruebas ──────────────────────────────────────────────────────────
72 personal = [
73  Docente("12345678", "Juan Carlos", "Mamani Quispe", 16, "Asociado"),
74  Docente("23456789", "María Elena", "Quispe Torres", 12, "Principal"),
75  Administrativo("34567890", "Pedro", "Huanca Lazo", 2800.0, 350.0),
76  Investigador("45678901", "Ana Lucía", "Flores Ramos", 2),
77 ]
78 planilla(personal)
79 # Verificar LSP — todos son PersonalUniversitario
80 for p in personal:
81  assert isinstance(p, PersonalUniversitario)
82  assert p.calcular_remuneracion() > 0
83 print("\nVerificación LSP: OK — todos los objetos respetan el contrato
   base")
```

5.3. Actividad 3 — Verificación del LSP — Test de sustitución

Implemente la función verificar_lsp() que, dado cualquier objeto Personal Universitario, ejecuta un contrato invariante sobre él. Luego demuestre que TODAS las subclases lo pasan, Finalmente, implemente la jerarquía Figura –> Rectangulo –> Cuadrado y explique por qué Cuadrado VIOLA el LSP.

```
1 # ── Verificación LSP para la jerarquía PersonalUniversitario ──────────
2 def verificar_contrato_personal(p: PersonalUniversitario) -> bool:
3     """
4     Contrato que TODA subclase debe satisfacer:
5     1. nombre_completo retorna string no vacío
6     2. calcular_remuneracion() retorna valor > 0
7     3. str(p) contiene el nombre completo
8     """
9     ok = True
10    ok = ok and isinstance(p.nombre_completo, str) and len(p.nombre_completo) > 0
11    ok = ok and p.calcular_remuneracion() > 0
12    ok = ok and p.nombre_completo in str(p)
13    return ok
14
15
16 def verificar_lsp(personal: list[PersonalUniversitario]) -> None:
17     """Ejecuta el contrato sobre cada objeto y muestra resultados."""
18     for p in personal:
19         if verificar_contrato_personal(p):
20             print(f"OK → {p.nombre_completo}")
21         else:
22             print(f"FALLA → {p.nombre_completo}")
23
24
```

```python
# ── Pruebas con la jerarquía PersonalUniversitario ──────────────────────
personal = [
    Docente("12345678", "Juan Carlos", "Mamani Quispe", 16, "Asociado"),
    Docente("23456789", "María Elena", "Quispe Torres", 12, "Principal"),
    Administrativo("34567890", "Pedro", "Huanca Lazo", 2800.0, 350.0),
    Investigador("45678901", "Ana Lucía", "Flores Ramos", 2),
]

verificar_lsp(personal)
print("\nVerificación LSP: OK — todos los objetos respetan el contrato base")


# ── Violación clásica de LSP: Cuadrado hereda de Rectángulo ─────────────
class Figura:
    def area(self) -> float: ...


class Rectangulo(Figura):
    def __init__(self, ancho: float, alto: float):
        self._ancho = ancho
        self._alto = alto

    def set_ancho(self, v: float) -> None: self._ancho = v
    def set_alto(self, v: float) -> None: self._alto = v
    def area(self) -> float: return self._ancho * self._alto


class Cuadrado(Rectangulo):
    def __init__(self, lado: float):
        super().__init__(lado, lado)

    def set_ancho(self, v: float) -> None:
        self._ancho = v
        self._alto = v  # fuerza ambos lados iguales

    def set_alto(self, v: float) -> None:
        self._ancho = v
        self._alto = v  # fuerza ambos lados iguales


def test_rectangulo(r: Rectangulo) -> None:
    """Test que DEBERÍA pasar para cualquier Rectangulo según LSP."""
    r.set_ancho(4)
    r.set_alto(5)
    area_esperada = 20.0
    assert r.area() == area_esperada, f"LSP violado: area={r.area()},
esperado={area_esperada}"


# ── Pruebas con Rectángulo y Cuadrado ──────────────────────────────────
rect = Rectangulo(2, 2)
test_rectangulo(rect)  #  OK

cuad = Cuadrado(2)
try:
    test_rectangulo(cuad)  #  FALLA — viola LSP
except AssertionError as e:
```

```
    print(f"LSP VIOLADO en Cuadrado: {e}")
    print("Solución: no heredar Cuadrado de Rectángulo; usar composición o interfaz independiente.")
```

Pregunta de Análisis

¿Por qué Cuadrado viola el LSP aunque matemáticamente un cuadrado ES un rectángulo?

Cuadrado viola el LSP porque cambia las reglas de la clase base Rectángulo y rompe el contrato esperado por cualquier código que trabaje con rectángulos.

¿Qué alternativa de diseño propone modelar ambas figuras sin violar el principio?

La interfaz común es la mejor opción porque mantiene la jerarquía limpia, asegura que cada clase cumpla su propio contrato y permite el polimorfismo sin violar el LSP. La composición es válida, pero suele ser más útil cuando quieres reutilizar lógica interna; para este caso, la interfaz es más clara y elegante.

5.4. Actividad 4 — Jerarquía en C++17 — virtual, override y destructor

Implemente en C++17 la jerarquía de personal universitario equivalente a la Actividad 2. Debe incluir: destructor virtual en la base, método virtual puro calcularRenumeracion(), polimorfismo con vector de unique_ptr, y la función planilla()

```cpp
// ── personal_unap.hpp — Práctica S3, Actividad 4 ──────────────────────
#pragma once
#include <string>
#include <vector>
#include <memory>
#include <stdexcept>
#include <iostream>
#include <iomanip>

// Clase base abstracta
class PersonalUniversitario {
protected:
    std::string dni_;
    std::string nombres_;
    std::string apellidos_;
public:
    PersonalUniversitario(std::string dni,
                          std::string nombres,
                          std::string apellidos)
        : dni_(std::move(dni)),
          nombres_(std::move(nombres)),
          apellidos_(std::move(apellidos)) {}
```

```cpp

    virtual ~PersonalUniversitario() = default; // destructor virtual OBLIGATORIO

    // Método virtual puro: contrato que TODAS las subclases deben implementar
    virtual float calcularRemuneracion() const = 0;

    virtual std::string identificarse() const {
        return apellidos_ + ", " + nombres_ + " (DNI: " + dni_ + ")";
    }
};

// ── Subclase Docente ──────────────────────────────────────────
class Docente : public PersonalUniversitario {
    int horas_;
    float tarifa_ = 75.0f;
    float factor_;

    static float resolverFactor(const std::string& cat) {
        if (cat == "Auxiliar") return 1.0f;
        if (cat == "Asociado") return 1.4f;
        if (cat == "Principal") return 1.8f;
        throw std::invalid_argument("Categoría inválida: " + cat);
    }
public:
    Docente(std::string dni, std::string nombres, std::string apellidos,
            int horas, const std::string& categoria)
        : PersonalUniversitario(std::move(dni), std::move(nombres),
                                std::move(apellidos)),
          horas_(horas), factor_(resolverFactor(categoria)) {
        if (horas <= 0 || horas > 40)
            throw std::invalid_argument("Horas fuera de rango [1,40]");
    }

    float calcularRemuneracion() const override {
        return horas_ * tarifa_ * factor_;
    }
};

// ── Subclase Administrativo ───────────────────────────────────
class Administrativo : public PersonalUniversitario {
    float salarioBase_;
    float bono_;
public:
    Administrativo(std::string dni, std::string nombres, std::string apellidos,
                   float salarioBase, float bono = 0.0f)
        : PersonalUniversitario(std::move(dni), std::move(nombres),
                                std::move(apellidos)),
          salarioBase_(salarioBase), bono_(bono) {
        if (salarioBase_ <= 0)
            throw std::invalid_argument("Salario base debe ser positivo");
    }

    float calcularRemuneracion() const override {
        return salarioBase_ + bono_;
    }
};

```

```cpp
// ── Subclase Investigador ──────────────────────────────────────
class Investigador : public PersonalUniversitario {
    int proyectos_;
    static constexpr float REMUNERACION_PROYECTO = 1500.0f;
public:
    Investigador(std::string dni, std::string nombres, std::string apellidos,
                 int proyectos)
        : PersonalUniversitario(std::move(dni), std::move(nombres),
                                std::move(apellidos)),
          proyectos_(proyectos) {
        if (proyectos < 1)
            throw std::invalid_argument("Debe tener al menos 1 proyecto activo");
    }

    float calcularRemuneracion() const override {
        return proyectos_ * REMUNERACION_PROYECTO;
    }
};

// ── Función planilla() — demuestra polimorfismo dinámico ──────
void planilla(const std::vector<std::unique_ptr<PersonalUniversitario>>& p) {
    float total = 0.0f;
    std::cout << std::string(60, '=') << "\n";
    std::cout << "PLANILLA PERSONAL UNIVERSITARIO — UNAP PUNO\n";
    std::cout << std::string(60, '=') << "\n";
    for (const auto& persona : p) {
        std::cout << std::fixed << std::setprecision(2)
                  << persona->identificarse()
                  << " S/ " << persona->calcularRemuneracion() << "\n";
        total += persona->calcularRemuneracion();
    }
    std::cout << std::string(60, '-') << "\n";
    std::cout << "TOTAL: S/ " << total << "\n";
}
```

5.5. Actividad 5 — Proyecto Integrador — Jerarquía de herencia del sistema

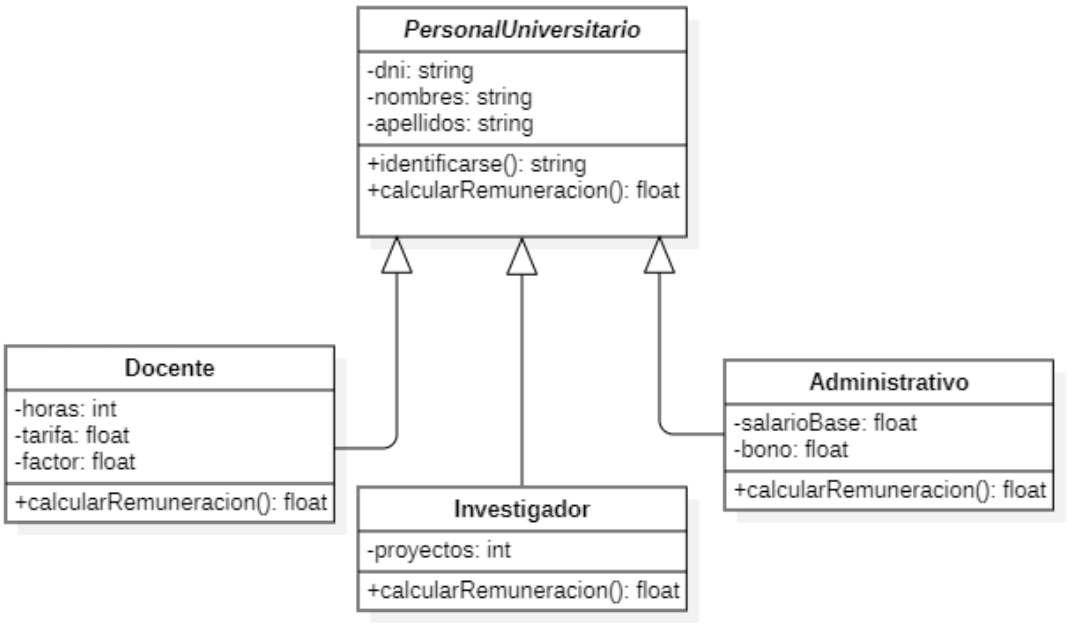
Aplique herencia al Proyecto Integrador definido en las Semanas 1 y 2. Esta actividad extiende el código ya entregado.

Identifique y justifique al menos 1 clase base y 2 subclases IS-A en su dominio. Complete la tabla

Clase Base	Subclase 1	Subclase 2	Relación IS-A verificada
PersonalUniversitario	Docente	Administrativo	Un Docente es un PersonalUniversitario;

			un Administrativo es un PersonalUniversitario

6. Diagrama UML



7. Preguntas de Reflexión

1. En su Proyecto Integrador, ¿encontró alguna situación donde pensó en usar herencia pero al final eligió composición (o viceversa)? Describa el caso y justifique la decisión.

   En el desarrollo del Proyecto Integrador, sí encontré una situación donde inicialmente pensé en aplicar herencia, pero finalmente opté por composición. El caso fue la relación entre la clase Informe y la clase Impresora. Al inicio parecía razonable modelar que un Informe “heredara” de Impresora, ya que ambos estaban relacionados con la acción de imprimir. Sin embargo, al analizar el dominio, me di cuenta de que un informe no es un tipo de impresora; son entidades de naturaleza distinta. El informe es un documento, mientras que la impresora es un dispositivo.

Por esa razón, la relación correcta fue HAS-A (composición): un objeto Informe puede tener una referencia a una Impresora para ejecutar la acción de imprimir, pero no hereda de ella. La decisión se justifica porque la herencia debe usarse únicamente cuando existe una relación IS-A clara y verificable, mientras que en este caso lo que existe es una relación de uso o dependencia.

Este ejemplo me permitió diferenciar mejor cuándo aplicar IS-A y cuándo aplicar HAS-A en el diseño orientado a objetos, evitando jerarquías frágiles y asegurando un modelo más coherente con la realidad del dominio.

2. ¿Por qué en C++ es obligatorio que el destructor de la clase base sea virtual si se usa polimorfismo dinámico? ¿Qué ocurre exactamente si no se declara virtual y se destruye un objeto a través de un puntero a la base?
   En C++, cuando se utiliza polimorfismo dinámico, es obligatorio que el destructor de la clase base sea virtual. La razón es que, al destruir un objeto a través de un puntero a la clase base, el compilador necesita saber cuál destructor ejecutar en tiempo de ejecución.

- Si el destructor es virtual, la llamada a delete basePtr; invoca primero el destructor de la subclase y luego el de la clase base, liberando correctamente todos los recursos.
- Si el destructor no es virtual, solo se ejecuta el destructor de la clase base. Esto provoca que los recursos propios de la subclase no se liberen, generando fugas de memoria o incluso comportamiento indefinido.

3. El polimorfismo en Python es "duck typing": cualquier objeto que tenga el método correcto puede usarse, sin necesidad de herencia formal. ¿Cuáles son las ventajas y desventajas de este enfoque frente al polimorfismo de C++ basado en tipos estrictos?
   En Python, el polimorfismo se basa en el principio de duck typing: “si camina como un pato y hace cuac como un pato, entonces es un pato”. Es decir, cualquier objeto que implemente el método esperado puede usarse, sin necesidad de herencia formal ni de declarar interfaces explícitas. En cambio, en C++ el polimorfismo dinámico depende de tipos estrictos y de la declaración explícita de métodos virtual en una jerarquía de clases.